From the Qn Universe to QnIR v0.2

Loop-2 adversarial review
Answers to QO-16–QO-32 and disposition of sixteen objections

Reviewed artifact
Nguyen, *From the Qn Universe to QnIR v0.2*, draft v0.2.2, 27 October 2026.

Author and maintainer
Christophe Duy Quang Nguyen — cdqn

Reviewer
Kimi K3.1-Pro (Kimi), AI-assisted adversarial review
27 October 2026

1. Scope and verdict

This review evaluates the supplied v0.2.2 PDF as a specification, not as an implementation. It answers QO-16–QO-32 in order, grades the sixteen anticipated objections, and adds defects not isolated in Section 14. Repository bytes, annex round trips, benchmarks, proof artifacts, and validator behavior were not independently reproduced; consequently, no conclusion below relies on their alleged success.

Overall verdict: major revision; do not promote QnIR v0.2 to the canon. The revision improves the v0.1 architecture in five respects: immutable values are preferable to mutable slots; static and dynamic commitments are separated; fatal and recoverable outcomes are distinguished; ceilings are made explicit; and external claims are treated as unverified inputs. These improvements do not yet yield a complete abstract machine. The remaining failures concern semantics and accounting, not exposition.

Four defects block the central claims. First, R1′ excludes runtime-dependent costs, yet the formulas for multiplication, exponentiation, containers, and verification use operand heights, exponents, sizes, or segment lengths. Second, the dispatch deducts a base cost and then reserves a table cost that already includes that base cost. Third, one value per static instruction, loop-carried block parameters, calls, and repeated execution have no coherent dynamic SSA semantics. Fourth, the proposed trace hash omits the resulting value and most machine state; it therefore cannot, by itself, commit to the execution whose determinism the receipt is said to certify.

The appropriate v0.3 milestone is therefore a small, executable core with a formal transition relation and differential tests. Geometric products, containers, calls, fork–join, and STARK proving should remain profile-gated extensions until each has a specified state transition, cost bound, and validator. The draft's narrow "no implementation, no superiority, no closure" disclaimer is accurate and should remain.

2. Blocking findings and minimal repairs

F1 — Static costs are not static

Finding. §5.2 and §8.3 violate R1′. OP_MUL uses h1,h2; OP_POW uses B; Class H uses M; OP_VERIFY uses K. Dense/sparse status is also asserted at load time although it depends on operand representation.

Required repair. Replace every runtime quantity by a declared immediate/header bound, or type each value with a static bound and define that bound as part of static semantics. Reject values that exceed their declared type bound before dispatch.

F2 — Base work is charged twice

Finding. §5.2 states that table costs include cbase=1. §8.2 stage 1 deducts cbase; stage 2 then reserves Cop, which includes it again.

Required repair. Define Cop as semantic work excluding dispatch and charge cbase+Cop once, or define Cop as the total and remove stage 1 as a separate deduction. Publish the ledger equation and one trace per opcode class.

F3 — SSA and control flow lack a dynamic model

Finding. G contains one value per static instruction, but a loop executes an instruction many times. Loop headers need ϕ/block arguments whose backedge operands occur later statically, contradicting the universal srck<PC check. PC modification also omits OP_BR_STATUS, OP_CALL, and OP_RET.

Required repair. Use basic blocks with explicit block arguments and edges. Define dynamic values as (static value id, iteration/instance id), and specify call frames, return values, branch targets, status consumption, and receipt ordering in the state tuple.

F4 — The static budget check is unspecified or unsound

Finding. Summing one reservation per instruction ignores repeated loop bodies and calls. Expanding every path may be exponential even when execution is bounded.

Required repair. Compute a maximum over bounded control-flow paths using declared trip/depth ceilings, or require Wbudget as a dynamic fuel bound and remove the claim that a simple sum certifies admission. Include gateway, frame, branch, and proof-verification charges.

F5 — The execution commitment binds too little

Finding. Equation (4) commits to word index, clock, and status. It omits input/output value commitments, store/root changes, remaining work, receipts, call/frame state, and transition versions. Equal traces under Eq. (4) need not denote equal executions.

Required repair. Commit each full transition: prev root, program root, step index, PC, decoded word or its hash, input/output IDs, pre/post state roots, work charged/refunded, clock, status, receipt IDs, semantics/profile versions. Use deterministic, domain-separated encoding.

F6 — Collision checking is insufficient

Finding. A shallow tuple-header comparison cannot detect different payloads with equal headers. “Constant-time O(k)” is also not constant-time when k varies.

Required repair. Compare full canonical encodings or retain a collision bucket of complete artifacts. On a genuine collision, quarantine both; never treat hash equality alone as value equality. State the constant-time profile separately from correctness.

F7 — Worked examples contradict the ISA

Finding. Example 2 advertises calls but contains no OP_CALL or OP_FACT. %hdr is not a defined 12-bit SSA reference. BR_STATUS needs both a token immediate and an offset, but only one aux field is defined. A fatal division error halts immediately, so a later OP_HALT cannot seal it.

Required repair. Re-encode every example with a normative assembler and validator. Add extension/header references, branch operands, exact call traces, and fatal-terminal traces to the conformance suite.

F8 — The isomorphism claim is too strong

Finding. Topological flattening of a DAG is not an isomorphism onto streams containing loops, status recovery, calls, or forks. Node-class coverage does not establish semantic preservation.

Required repair. Replace R11′ “isomorphism” with a defined compilation relation. Prove forward simulation and, where intended, backward simulation for a typed source and target semantics.

3. Answers to QO-16–QO-32

QO-16 — AIR arithmetization

Verdict. A single monolithic 49-opcode AIR is not the honest default. Specify a universal outer transition with class-specific sub-AIRs and explicit lookup/permutation arguments. Class D can dominate the trace; no prover-cost ranking is numerically decidable from the PDF.

The outer AIR should fix a 64-bit decoder, PC/next-PC relation, type and status rules, work ledger, and full pre/post-state commitments. Selector polynomials activate exactly one legal opcode relation per step. Shared value, store, receipt, and constant-table accesses use permutation or logarithmic-derivative lookup arguments; a lookup must prove both table membership and applicability of the selected rule, not merely that an output tuple

occurs somewhere in a table.

The 16-rule bypass table should be keyed by (rule id, canonical input schema, canonical output schema, side-condition result). Class H needs authenticated path-copy constraints, bounded Merkle paths of length at most $\log_B Mmax$, child-index constraints, and a separate memory-consistency argument for persistent roots. The simplest sound fork–join AIR uses the canonical sequentialization of the partial order; native parallel proving requires composable branch proofs and a join relation, not a larger sequential step table.

For Class D, represent a sparse blade operand as a sorted (frame id, blade index, coefficient) list. Geometric-product constraints must prove sign, blade-index XOR, coefficient multiplication, duplicate collection, and canonical zero removal. Dense dimension n incurs up to 2^n blades per operand; sparsity must therefore be a static type bound, not prose. STARK proving normally adds at least quasi-linear trace-processing overhead, but the break-even against Anchor-Cut depends on trace length, field, blowup, queries, hash implementation, and hardware. Publish those parameters and measured traces before selecting a default realization. [2]

QO-17 — Cost-polynomial calibration

Verdict. No finite benchmark suite, by itself, certifies a worst-case bound. A sound target-level bound must come from compositional worst-case analysis; measurements validate and calibrate it but do not prove universal domination.

The certificate chain should be: source ceiling → static QnIR transition bound → verified lowering rule → target instruction/sequence bound → memory and contention bound → hardware telemetry envelope. Each arrow needs a versioned artifact. Time and energy require separate coefficients: a sequence that is worst in cycles need not be worst in joules. The phrase “affine WCET/energy receipt” must identify the target, microarchitecture, clock, memory model, compiler, and uncertainty treatment.

For each opcode, publish adversarial worst-case operands, lowering traces, predicted U, measured cycles/energy, and residuals. Then test out-of-sample shapes at ceiling boundaries. Statistical margins may quantify measured uncertainty; they cannot convert an unbounded path into a certified bound. Correct F1–F4 before calibration, because calibration cannot rescue an invalid accounting relation.

QO-18 — Liveness window

Verdict. $Wlive \leq 256$ is not justified by the supplied evidence and should not be a universal validity limit. It is a target-profile register-allocation parameter.

Report static and dynamic liveness separately. For each kernel, publish the maximum live set over all bounded paths and the dynamic histogram per executed step, including p50, p95, p99, maximum, spill count, and spilled bytes. Include straight-line, loop-carried, call-heavy, forked, and container kernels. If a program exceeds a profile window, the correct outcomes are spilling, stream segmentation, or profile rejection—not rejection by the core IR. The 12-bit index ceiling of 4096 static values likewise needs a defined extension or module boundary.

QO-19 — Container lowering BOC

Verdict. Unresolved. Neither HAMT/RRB nor a Merkle B-tree can be selected from asymptotic update cost alone.

Use identical canonical serialization, hash function, persistence policy, cache model, and batch rules. Test insertion-dominated and scan-dominated workloads, plus point reads, version retention, and adversarial hash/key distributions. Report worst and measured ΔW, allocated nodes per update, read amplification, bytes hashed, cache misses, proof size, implementation coupling, and coverage of all five Category C types.

The lexicographic Tier-2 comparison must apply the declared order to measured distributions, not only means. If insertion and scanning select different winners, split the opcode contract by container type or make lowering profile-specific. The core ISA should specify persistent container semantics independently of either data structure.

QO-20 — Tree-clock receipt ordering

Verdict. Lexicographic branch-id merging is deterministic but does not, by itself, preserve causal birth order. Concurrent births have no intrinsic global temporal order; any total order must be an explicitly constructed topological extension.

The validator should require: parent-before-child within every branch; fork before descendants; descendants before join; and branch id used only to order causally unrelated sibling sequences. If Axiom 5 demands a global birth sequence, compute a stable topological sort with branch id and local index as tie-breakers. Do not claim that sorting sibling branch ids alone proves causal preservation.

Canonical κtree encoding must specify the clock algebra (event, fork, join), branch-id width and byte order, integer bounds, and domain separation. Deterministic CBOR should use definite lengths, canonical integer widths, sorted map keys, and no floating-point values. [3,4] The trace commitment must then include the complete clock value and full transition record of F5.

QO-21 — The call-pair trade

Verdict. Move OP_CALL/OP_RET to a profile-gated extension. The current byte calculation omits the stored callee body, and verification amortization is asserted without a verifier model.

If a P-word pattern is instantiated at m sites, inline storage is mP words; call-form storage, including one callee body, is P+m call-site words before return/frame overhead. The gross saving is (m−1)P−m words, so the simple storage break-even is $P>m/(m-1)$ for $m>1$, not the universal statement $P>2$. If the callee already resides in a shared content store, that storage assumption must be stated and its lookup/receipt cost charged.

Verification work can be amortized only if the callee is typed, context-independent, and its receipt is valid at the call site. Otherwise, inlining and verification remain per instantiation. Specify argument/return conventions, maximum frame bytes Fmax, Cmax·Fmax memory, receipt order, failure propagation, and sub-budget conservation. Retain calls only after a measured comparison includes these costs.

QO-22 — Token taxonomy

Verdict. The two-tier idea is sound; the present partition and mappings are not. A cache miss is operational control flow, not necessarily an epistemic defect. A negative even root is a domain/no-solution outcome, not evidence of excessive complexity.

STATUS_LOOKUP_MISS should be consumed internally by the specified recompute fallback and leave no terminal debt after successful recomputation. INCONCLUSIVE is appropriate only if the workflow declines or fails to obtain the requested evidence. STATUS_NEG_EVEN_ROOT should map to NO_SOLUTION in the real frame, unless the program explicitly promotes the frame; EXPRESSION_TOO_COMPLEX should denote resource/representation limits.

Define token lifecycle algebraically: birth, observation, retirement, propagation, and terminalization. Specify whether observation consumes a token, how successful recovery retires it, and which terminal wins when several values remain unresolved. OP_SELECT cannot by itself retire arbitrary semantic debt. Add conformance tests for success after recovery, unresolved debt at every exit, and multiple outstanding tokens.

QO-23 — Gateway fee fairness

Verdict. Use a metered, capped reservation with refunds, explicit authorization, and governance independent of the invoking principal. Do not make payment depend on whether validation succeeds.

A defensible schedule is Fmax=a0+Σj ajCj, where Cj are declared work ceilings for parsing, contract validation, anchor checks, and proof verification; a0 covers fixed admission work. Reserve Fmax, charge actual metered work, and refund the residue. Outcome-independent pricing preserves validator incentives, while authorization tokens, rate limits, and per-principal quotas prevent an attacker from inducing billable crossings.

The DCC profile may declare the formula and cap, but an open governance process must approve coefficient ranges and revisions. Receipts should itemize each fee component. A principal-set arbitrary fee is not a fairness mechanism; a success-contingent fee would instead reward favorable outcomes and bias validation.

QO-24 — Zoom normalization

Verdict. Keep minimal-zoom normalization for denotational value identity. Make zoom history mandatory in a separately committed artifact-lineage record. Do not conflate numeric identity with derivation identity.

Two routes to the same exact number should deduplicate as values; they must not deduplicate as experiments. The normative artifact record should contain the pre-normalization tuple, normalization version, zoom transitions, work charged/refunded, parent artifacts, and resulting canonical value ID. The normalizer needs a proof of unique minimal representation and deterministic behavior at lattice boundaries.

This separation resolves the stated erasure case without bloating every numeric key with history. Conservativity audits query the lineage commitment; value equality uses the normalized commitment.

QO-25 — Domain-tag registry

Verdict. Place allocation under Q(anchor) governance, with a public append-only registry and non-reuse rules. A registry alone does not prevent squatting; namespace control and challenge procedures are required.

Each entry should bind DomainTag to a canonical schema/name, governance key, version, status, registration receipt, and predecessor. Allocation should require proof of control of the governing namespace, uniqueness checking, and a dispute/appeal path. Revocation must not rewrite history and retired tags must never be reassigned.

For cross-swarm allocation without a central race, derive long tags as hashes of a canonical governed namespace and schema identifier, then register them for discoverability. Domain separation is effective only if encoders include the tag unambiguously and validators reject unknown or retired tags according to a versioned policy.

QO-26 — Choreographic projection

Verdict. Projection purity is necessary but insufficient. Trust requires local type correctness, communication compatibility, deadlock-freedom proofs, and a refinement theorem relating global and projected traces.

Define a global protocol type with explicit roles, messages, parent receipt IDs, budgets, and failure edges. Prove deterministic endpoint generation for a fixed compiler generation; local well-formedness; send/receive matching; bounded buffers; and progress under the declared fairness and failure assumptions. The central theorem is trace refinement: every permitted interleaving of endpoint traces corresponds, modulo canonical ordering, to a legal global trace, and conversely for completed global executions.

A mid-node fatal receipt must become an addressed lineage event for every blocked peer, with parent hashes and a unique protocol-instance ID. Retries need idempotency keys and fresh bounded budgets; otherwise silent retry loops reappear outside the local totality proof. A two-node example is a conformance test, not a proof of general deadlock freedom.

QO-27 — Verbalizer certification

Verdict. Bind the exact verbalizer generation and exact read-back text into the signed intent artifact. A version label alone is insufficient.

The contract should carry hashes of the verbalizer implementation or deterministic build artifact, contract schema, locale/terminology registry, generation parameters, and canonical output text. The principal co-signs that tuple. Verification recomputes the read-back or checks its committed text and rejects unknown generations. Regression vectors and independent semantic tests should accompany every generation change.

If V is statistical, deterministic generation cannot be inferred from a signature; such output remains a hypothesis until human/formal anchoring. Thus H(T|Contract)=0 must be restated as deterministic read-back relative to a committed V generation and schema, not as a property of an unversioned V.

QO-28 — Saturation determinism boundary

Verdict. Distinguish value identity, artifact identity, and execution identity. Extractor generation belongs in derivation identity and lineage; it need not alter numeric value identity when canonical results are equal.

Define drift relative to a fully pinned tuple: intent, dependencies, ISA/semantics version, schedule, ceilings, extractor generation, profile, and inputs. If only extractor generation changes and the artifact declares that change, the result is measured divergence, not state drift. Undeclared changes, cache-key collisions, or unequal outputs under a pinned tuple are drift or implementation failure.

Artifact/cache keys should include extractor generation and all semantic inputs. A shared workflow may accept different generations only through an explicit compatibility rule: identical canonical output, a proof of equivalence, or a gateway-approved version policy. A lineage block alone is inadequate unless the generation participates in cache and receipt verification.

QO-29 — Falsifiable positioning

Verdict. No counterexample is established by this review, but absence of a named counterexample does not establish the composition claim. Retain it as a falsifiable research hypothesis.

The proposed acceptance clause—“accept that the composition claim stands until one exists”—reverses the burden of proof. Replace it with a dated systematic comparison protocol. For each system and version, give executable or normative evidence for columns (a)–(e), identify required extensions, and distinguish a single integrated artifact from a manually assembled stack.

The contribution may remain valid even if another system satisfies all five columns: implementation simplicity, auditability, or measured cost could differentiate it. Novelty should therefore be claimed only after the comparison, while the joint design objective can be stated now without qualification.

QO-30 — Flags pressure

Verdict. Four bits cannot be certified as future-proof. Preserve the 64-bit word, but add a deterministic per-instruction extension side table rather than forcing all attributes into per-program header flags.

Header promotion is valid only for attributes uniform across the program. For instruction-local overflow, reserve a flag pattern as EXT and use aux or the PC to index a canonical header extension record containing the additional attributes. This preserves stream alignment and avoids a second word format. Alternatively, widen

aux in a versioned new ISA, but do not pretend that this is the same encoding.

The validator should report flag utilization by opcode and extension-record counts by program. Unknown flag combinations must be rejected. Fixed-width instructions are helpful to decoding, but AIR tractability comes from explicit selectors and tables, not from a prohibition on side tables.

QO-31 — External-claims intake

Verdict. The constitutional disposition is correct; the proposed automatic recalibration is too strong. A verified ETH refutation changes no stated QnIR axiom or transition rule. Guidance should change only after relevant workloads or lowering costs are actually measured.

The source repository confirms the 721-manuscript/373-family catalogue and describes the artifacts as raw research data at differing verification stages. [5] OpenAI's own release records the ETH and local-smoothing claims and their Lean artifacts; the external ETH group's examination is not completed independent certification. [6,7] These claims must therefore remain external observations, not premises in this review.

The lexical scan is useful evidence of non-dependence but not a proof: concepts can be implicit under different terminology. A citation walk and an assumption inventory should accompany it. NP-hardness does not disappear if ETH fails. Finally, even a subexponential 3-SAT algorithm need not improve practical saturation workloads; asymptotic refutation and measured engineering cost are distinct.

QO-32 — Evidence-cluster receipting

Verdict. Keep evidence-cluster analysis at validator level. Do not infer shared derivation from "overlapping H(Trace) roots," and do not add such a field to every receipt on that rationale.

A single Merkle root does not overlap another root. Equal roots can identify identical committed subtraces only when the relevant nodes, membership proofs, and schemas are available. Shared use of a method need not produce any identical subtrace; conversely, identical library subtraces do not imply that two conclusions are epistemically dependent. The proposed hash test therefore has both false negatives and false positives.

Use an explicit claim–evidence DAG: claim ID, evidence IDs, derivation-artifact IDs, parent claims, method family, and independence rationale. Validators may compute cluster diagnostics from that graph and expose compact cluster commitments on demand. Independence remains a modeled judgment, not a hash equality. This preserves anti-double-counting without imposing per-receipt bytes or pretending that lineage similarity settles evidential independence.

4. Disposition of the sixteen anticipated objections

Grading convention: **fatal** blocks promotion to a normative v0.3 specification; **cosmetic** requires local wording or bookkeeping repair; **surviving** means the design withstands the objection when its stated scope is enforced. Scope qualifications are explicit.

Objection	Grade	Disposition
1	Surviving, conditionally	SSA is substantive only after block arguments and dynamic instances are formalized; otherwise the criticism is correct.
2	Fatal	No evidence supports 4096 births or 256 live values; overflow behavior is unspecified.
3	Fatal	The objection is confirmed and strengthened by F1, F2, and F4.
4	Cosmetic	Refunding unused reservations creates no units if gross reserve, actual charge, refund, and net charge are separate ledger fields. Breakeven logic must use comparable actual costs.

Objection	Grade	Disposition
5	Fatal	Lifecycle and consumption are undefined; lookup-miss and negative-root mappings are semantically wrong.
6	Fatal	Frame state, return convention, verification reuse, and transitive budgets are not specified.
7	Fatal for canonization	Container semantics may remain experimental, but a lowering winner cannot be canonized before the BOC evidence.
8	Fatal	Data disjointness needs a load-time proof relation or certified analysis. Declaring it is insufficient.
9	Fatal	No AIR exists and Class D may dominate proving cost. “Arithmetizes directly” must be removed until demonstrated.
10	Surviving at IR level	The fee is governance metadata, not a semantic totality defect; deployment remains blocked until fairness and anti-griefing rules exist.
11	Surviving with repair	Value identity and derivation identity solve the problem, but lineage must be normative rather than a note.
12	Cosmetic	Origin quotienting is not a privacy guarantee. Add a linkability threat model and consumer obligations; do not claim anonymity.
13	Surviving	The draft correctly limits the certificate to a principal-anchored intent. Cryptography cannot certify informed consent.
14	Surviving as hypothesis	The five-column goal is legitimate; established joint novelty is unproven and the burden remains on the claimant.
15	Fatal	A signed read-back is not stable unless the exact verbalizer generation and text are committed.
16	Surviving, unverified here	The proposed independent-extractor check is appropriate. This review did not reproduce it and therefore assigns no fidelity pass.

5. Required v0.3 repair sequence

The order matters because later artifacts inherit earlier definitions.

Step	Action	Acceptance condition
1	Freeze a minimal core	Constants, tuples, exact arithmetic, comparison, bounded structured control, status, commit, witness, and halt. Move calls, saturation, containers, fork–join, and road proofs to extensions.
2	Publish operational semantics	Typed configurations, small-step transitions, block arguments, dynamic instances, token lifecycle, failure behavior, and deterministic encodings. State determinism as Eval(Intent,D,V)=Execution with all semantic versions explicit.
3	Repair work accounting	One ledger equation; static operand bounds; sound control-flow budget admission; reserve/charge/refund invariants; no double billing.
4	Build a conformance oracle	Assembler, validator, interpreter, and independent second implementation. Re-encode all examples and add adversarial tests for loops, calls, collisions, unresolved statuses, and budget boundaries.
5	Strengthen commitments	Full-transition trace roots, collision-safe artifact comparison, normative lineage, and versioned verbalizer/extractor identities.

Step	Action	Acceptance condition
6	Add extensions with evidence	Container BOC, call trade study, liveness histograms, choreographic refinement, then class-specific AIRs and measured Realization A/B break-even.

Promotion criterion: v0.3 may enter the canon only when the core semantics and ledger pass independent differential execution, every normative example validates, and each retained extension has an explicit proof or measurement artifact. Until then, “Candidate 1” must not be called Theorem 1, and R11′, R12′, R19, and the O(polylog) verification statement must remain conditional.

6. Conclusion

QnIR v0.2 is a coherent research direction but not yet a coherent machine specification. Its strongest methodological choice is the exposure of assumptions and pre-committed revisions. Its weakest choice is to convert architectural intentions into opcode tables before the corresponding state transitions and evidence exist. The next iteration should reduce scope, define the core relation exactly, and make every example mechanically checkable. This is the shortest route from a receipt vocabulary to a receipt-producing system.

References

[1] Nguyen, C. D. Q. (2026). *From the Qn Universe to QnIR v0.2* (Draft v0.2.2). CDQN repository. Reviewed PDF, including Annexes A–U.

[2] Ben-Sasson, E., Bentov, I., Horesh, Y., & Riabzev, M. (2018). *Scalable, transparent, and post-quantum secure computational integrity*. IACR Cryptology ePrint Archive, 2018/046. https://eprint.iacr.org/2018/046

[3] Almeida, P. S., Baquero, C., & Fonte, V. (2008). Interval tree clocks. In *Proceedings of PPoPP 2008*. https://doi.org/10.1145/1345206.1345230

[4] Bormann, C., & Hoffman, P. E. (2020). *Concise Binary Object Representation (CBOR)* (RFC 8949), especially deterministic encoding. https://www.rfc-editor.org/rfc/rfc8949

[5] OpenAI. (2026). *openai/math: Mathematical manuscripts and supporting proof artifacts* [Repository and README]. https://github.com/openai/math

[6] OpenAI. (2026). *History* [Repository release record, entries of 26 October 2026]. https://github.com/openai/math/blob/main/history.md

[7] OpenAI. (2026, October 27). *Working with academic communities to build scientific understanding*. https://openai.com/index/building-scientific-understanding-with-academic-communities/

Review limitation. This is an AI-assisted technical review of the supplied artifact. It is not human peer review, a security audit, a correctness proof, or independent reproduction of the repository.